

CSE 719 — Paxos Solutions

Lecture-07 · Exam: Wed 1 Jul 2026 · Dr. Atiqur Rahman

Contents

1	Reference: Paxos Protocol	2
1.1	Consensus Problem	2
1.2	Paxos Overview	2
1.3	Three Phases Per Round	2
2	2024 Q2a — Consensus Definition & Logical vs Physical Concurrency (1.5 marks)	3
3	2021 Q8c / 2024 Q7b — Leader Waits for Less Than Majority (2.5 / 3 marks)	4
4	2022 Q2b — Same Proposal Numbers + Strict-Smaller Reject Rule (6 marks)	5
5	2024 Q7a — Why $2f+1$ Nodes Required to Tolerate f Failures (3 marks)	6
6	2024 Q7c — Primary-Backup vs Paxos: Three Scenarios (3 marks)	6
7	Summary Table	7

Reference: Paxos Protocol

Consensus Problem

Definition: N processes each hold a private input (0 or 1) and an output variable set at most once. A consensus algorithm must guarantee that at termination *all correct processes* output the same value.

Three properties required:

- **Validity** — the decided value must have been proposed by some process.
- **Agreement** — no two correct processes decide different values.
- **Non-triviality** — both outcomes (0 and 1) are reachable from some initial state.

Key fact: Consensus is possible in synchronous systems but **impossible in asynchronous systems** (Fischer-Lynch-Paterson / FLP, 1985).

Consensus is *equivalent to or harder than*: perfect failure detection, leader election, and total-order multicast.

Paxos Overview

- Invented by **Leslie Lamport**. Used in Apache ZooKeeper (Yahoo!), Google Chubby, Active Disk Paxos.
- Does *not* solve the full consensus problem (impossible by FLP), but provides:
 - **Safety** — two processes never decide different values.
 - **Eventual Liveness** — if network conditions eventually stabilise, consensus will be reached. No guarantee on *when*.
- Runs in asynchronous **rounds**; each round has a unique **ballot id**. Higher ballot id = newer round. On hearing a message from round $j+1$ while in round j , abort everything and join round $j+1$.

Three Phases Per Round

Phase	Name	Action
1	Election	Candidate broadcasts ballot id; processes vote OK to highest seen
2	Proposal (Bill)	Leader sends proposed value; acceptors log and respond OK
3	Decision (Law)	On majority OKs, leader broadcasts final value; all log decision

Phase 1 — Election:

1. Candidate picks ballot id n strictly higher than any seen; broadcasts $\text{PREPARE}(n)$.
2. Each process: if $n > \text{highest seen ballot id} \Rightarrow$ respond OK and include any previously accepted value (with its ballot id). Log n to disk. Otherwise reject.
3. If candidate receives OKs from a **majority** $\Rightarrow$ it is the leader for this round. Adopt the value v' from the highest-ballot-id OK response (if any); otherwise use own value.

Phase 2 — Proposal:

1. Leader sends $\text{ACCEPT}(n, v)$ to all. Each acceptor logs and responds OK.

Phase 3 — Decision:

1. Leader receives OKs from majority $\Rightarrow$ broadcasts $\text{DECIDE}(v)$. All processes log decision.

Point of no-return: When a majority of processes have accepted value v in Phase 2 (responded OK), consensus is implicitly reached — even before the leader knows. Any subsequent round either chooses the same value or fails, because **any two majority sets intersect**, so the next leader's Phase 1 quorum will always contain at least one process carrying v' .

Fault handling:

- Process crash: excluded from quorum; on restart, reads log to recover past ballot ids and accepted values.
- Leader crash: another process starts a new round with higher ballot id.
- Message drop: timeout $\Rightarrow$ start new round.
- Protocol may never terminate (FLP); eventual liveness only.

Fault tolerance math:

- Majority = $\lfloor N/2 \rfloor + 1$ processes must respond.
- After f failures: need $N - f \geq \lfloor N/2 \rfloor + 1 \Rightarrow N \geq 2f + 1$.

2024 Q2a — Consensus Definition & Logical vs Physical Concurrency (1.5 marks)

Question: Define consensus algorithms. Differentiate between logical and physical concurrency.

Consensus Algorithm

A **consensus algorithm** is a distributed protocol that enables a set of processes — each starting with a private input value — to agree on a single common output value, satisfying agreement (no two correct processes decide different values), validity (the decided value was proposed by some process), and termination under partial synchrony.

Examples: Paxos (safety + eventual liveness), Raft, PBFT.

Logical vs Physical Concurrency

	Logical Concurrency	Physical Concurrency
Definition	Multiple tasks <i>interleaved</i> on a single CPU via time-multiplexing; appear concurrent	Multiple tasks execute <i>truly simultaneously</i> on separate CPUs
Parallelism	No (one instruction at a time)	Yes (instructions overlap in real time)
Hardware	Single processor	Multiple processors or cores
Example	OS thread scheduler on one core	Two transactions executing at the same instant on different servers

In distributed systems, **physical concurrency** is the norm: independent servers execute simultaneously with no global clock, creating race conditions that require concurrency control (locking, timestamps, consensus protocols like Paxos).

2021 Q8c / 2024 Q7b — Leader Waits for Less Than Majority (2.5 / 3 marks)

Question: In a Paxos system, the leader proceeds to Phase 2 after receiving OKs from *fewer than a majority* of acceptors in Phase 1. How does this break Paxos guarantees?

Why Majority is Necessary

Paxos safety rests on one invariant: **any two quorums (majority sets) must intersect**. This intersection ensures a newly-elected leader in Phase 1 always hears from at least one process that witnessed the previously committed value v' , preventing it from proposing a conflicting value.

If the leader proceeds with $k < \lfloor N/2 \rfloor + 1$ OKs in Phase 1, two quorums of size k can be **disjoint** when $k \leq N/2$. The intersection guarantee is lost.

Concrete Counterexample

System: 5 acceptors $\{A_1, A_2, A_3, A_4, A_5\}$. True majority = 3. Leader waits for $k = 2$.

Round 1, Leader L_1 (ballot 1):

- Phase 1 OKs from $\{A_1, A_2\}$ — L_1 proceeds (only 2 needed).
- Phase 2: proposes v_1 ; OKs from $\{A_1, A_2\}$. L_1 decides v_1 and multicasts.

Round 2, Leader L_2 (ballot 2):

- Phase 1 OKs from $\{A_3, A_4\}$ — *disjoint* from Round 1 set.
- A_3 and A_4 never heard of v_1 ; both respond with no prior value.
- L_2 proposes own value $v_2 \neq v_1$. Phase 2 OKs from $\{A_3, A_4\}$. L_2 decides v_2 .

Result: v_1 and v_2 are both decided — **safety is violated**.

Root Cause and Conclusion

With $k < \lfloor N/2 \rfloor + 1$, two leaders can collect Phase 1 quorums from completely disjoint sets. The new leader has no witness to the previously committed value, so it picks a different value and commits it. The intersection property — the entire foundation of Paxos safety — requires strict majority ($> N/2$) everywhere.

The leader must wait for $\lfloor N/2 \rfloor + 1$ OKs in Phase 1. Any smaller quorum breaks safety.

2022 Q2b — Same Proposal Numbers + Strict-Smaller Reject Rule (6 marks)

Question: Consider a modified Paxos where (i) multiple proposers may use the *same* ballot number, and (ii) acceptors reject a ballot only if it is *strictly smaller* than the highest seen (i.e., they accept a ballot id *equal to* what they have already processed). Is this correct? Provide a counterexample if not.

Analysis

Standard Paxos requires: (1) each proposer picks a *unique* ballot id, and (2) acceptors reject if new ballot id $\leq$ highest seen (i.e., accept only if *strictly greater*).

The modification relaxes both constraints. **This is not correct.** It allows two concurrent leaders to each achieve Phase 1 quorums with identical ballot ids, then commit conflicting values.

Counterexample

System: 3 acceptors $\{P_1, P_2, P_3\}$. Proposers L_A and L_B independently choose ballot id $n = 5$.

Phase 1:

- L_A sends PREPARE(5). P_1 and P_2 each respond OK (first time seeing ballot 5). L_A has quorum $\{P_1, P_2\}$.
- L_B sends PREPARE(5). P_2 responds OK (ballot 5 = 5, not strictly less, so accepted under modified rule). P_3 responds OK. L_B has quorum $\{P_2, P_3\}$.
- Neither leader heard any previously accepted value, so both choose their own values.

Phase 2:

- L_A sends ACCEPT(5, v_A). P_1 and P_2 both log and accept v_A .
- L_B sends ACCEPT(5, v_B) with $v_B \neq v_A$. P_2 accepts (ballot 5 = 5 $\Rightarrow$ not strictly less $\Rightarrow$ accepted). P_3 accepts. L_B commits v_B .

Outcome:

- $\{P_1, P_2\}$ accepted v_A ; $\{P_2, P_3\}$ accepted $v_B \neq v_A$.
- P_2 accepted *two different values* for the same ballot id $n = 5$.
- **Safety is violated:** two distinct values committed in the same Paxos instance.

Root Cause

In standard Paxos, accepting a ballot *strictly greater* than any seen ensures that once an acceptor votes for $n = 5$ from L_A , it rejects any subsequent $\text{PREPARE}(5)$ from L_B (since $5 \not> 5$). This mutual exclusion ensures at most one leader can form a valid Phase 1 quorum per ballot number.

Allowing equal ballot ids removes this mutual exclusion: the same ballot number can be used by two leaders simultaneously to achieve overlapping quorums, each unaware of the other's decision, breaking the invariant that a single value is committed per round.

2024 Q7a — Why $2f + 1$ Nodes Required to Tolerate f Failures (3 marks)

Question: Explain why Paxos cannot tolerate f node failures with fewer than $2f + 1$ total nodes.

Answer

Every Paxos operation (Phase 1 and Phase 2) requires a **majority** ($> N/2$) of all N nodes to respond. After f failures, $N - f$ nodes remain. For progress to be possible:

$$N - f > \frac{N}{2} \implies \frac{N}{2} > f \implies N > 2f \implies \boxed{N \geq 2f + 1}$$

With $N = 2f$ (insufficient):

- f nodes fail $\Rightarrow f$ nodes remain.
- Majority of $2f$ nodes = $f + 1$.
- $f < f + 1 \Rightarrow$ remaining nodes cannot form a majority $\Rightarrow$ **protocol stalls forever**.

With $N = 2f + 1$ (minimum sufficient):

- f nodes fail $\Rightarrow f + 1$ nodes remain.
- Majority of $2f + 1$ nodes = $f + 1$.
- $f + 1 = f + 1 \Rightarrow$ remaining nodes *exactly* form a majority $\Rightarrow$ **protocol can proceed**.

Safety also requires $2f + 1$: Any two majority sets chosen from $N = 2f + 1$ nodes must intersect. By pigeonhole:

$$|Q_1| + |Q_2| = (f + 1) + (f + 1) = 2f + 2 > 2f + 1 = N \Rightarrow |Q_1 \cap Q_2| \geq 1$$

This intersection carries the previously committed value v' into every future Phase 1, preserving safety. With $N = 2f$, two quorums of size $\lceil (2f)/2 \rceil = f$ could be disjoint ($f + f = 2f = N$), breaking safety.

Conclusion: $N \geq 2f + 1$ is the minimum required for both liveness (majority reachable after f failures) and safety (any two quorums intersect).

2024 Q7c — Primary-Backup vs Paxos: Three Scenarios (3 marks)

Question: For each scenario, choose primary-backup or Paxos and justify: (i) stock exchange lock server, (ii) documentary movie storage, (iii) login server.

Scenario	Choice	Justification
(i) Stock exchange server	Paxos	Correctness is critical: granting the same lock to two clients (due to split-brain or stale failover) causes direct financial harm. Primary-backup risks the backup granting a lock the primary already committed before crashing. Paxos consensus ensures a lock grant is agreed upon by a majority before any client is notified — split-brain is structurally impossible.
(ii) Documentary movie storage	Primary-backup	Movies are written once and read many times; there is no concurrent modification. Paxos imposes 3-phase overhead per write with zero correctness benefit here. Simple replication (primary streams to replicas) suffices; brief failover downtime is acceptable for archival storage.
(iii) Login server	Primary-backup	Authentication is read-heavy and requests are stateless between sessions. A brief downtime during failover (primary → backup) is acceptable — users re-submit login. Paxos overhead on every authentication attempt is unjustified; simple failover preserves availability with minimal complexity.

Decision rule:

- **Use Paxos** when correctness is absolute and split-brain is catastrophic (financial, legal, lock management).
- **Use primary-backup** when write-once or read-heavy, brief downtime is acceptable, and Paxos latency overhead is not justified.

Summary Table

Year	Q	Marks	Core answer
2024	Q2a (partial)	1.5	Consensus = all agree on one value; logical = interleaved, physical = si
2021	Q8c	2.5	$k < \text{majority} \Rightarrow \text{disjoint quorums possible} \Rightarrow \text{two values decided} \Rightarrow \text{saf}$
2024	Q7b	3	Same as 2021 Q8c (verbatim repeat)
2022	Q2b	6	Same ballot id $\Rightarrow$ two leaders both achieve quorum $\Rightarrow$ both commit dif
2024	Q7a	3	$N - f > N/2 \Rightarrow N \geq 2f + 1$; with $2f$ nodes, f remaining cannot form
2024	Q7c	3	Lock server $\rightarrow$ Paxos; Movie storage $\rightarrow$ P-B; Login server $\rightarrow$ P-B
Total		19	

Five facts to know cold:

1. Paxos provides **safety + eventual liveness**; NOT guaranteed termination (FLP).
2. **Any two majority sets always intersect** — the foundation of safety.
3. Majority = $\lfloor N/2 \rfloor + 1$; need $N \geq 2f + 1$ to tolerate f failures.
4. Leader < majority in Phase 1 $\Rightarrow$ disjoint quorums possible $\Rightarrow$ safety violated.
5. Same ballot id from two proposers $\Rightarrow$ both achieve quorum $\Rightarrow$ safety violated.